

图解 · 彻底理解 MLA 的计算过程



歪门正道
清华软件学院/AI infra

340 人赞同了该文章 ›

最近在工作中惊讶的发现，我好像已经很久没摸过“传统”[Attention⁺](#)了！不管是改 [KV cache⁺](#) 管理，还是上 Attention 的什么变体，一上来就是看看能不能适配 [MLA⁺](#)。

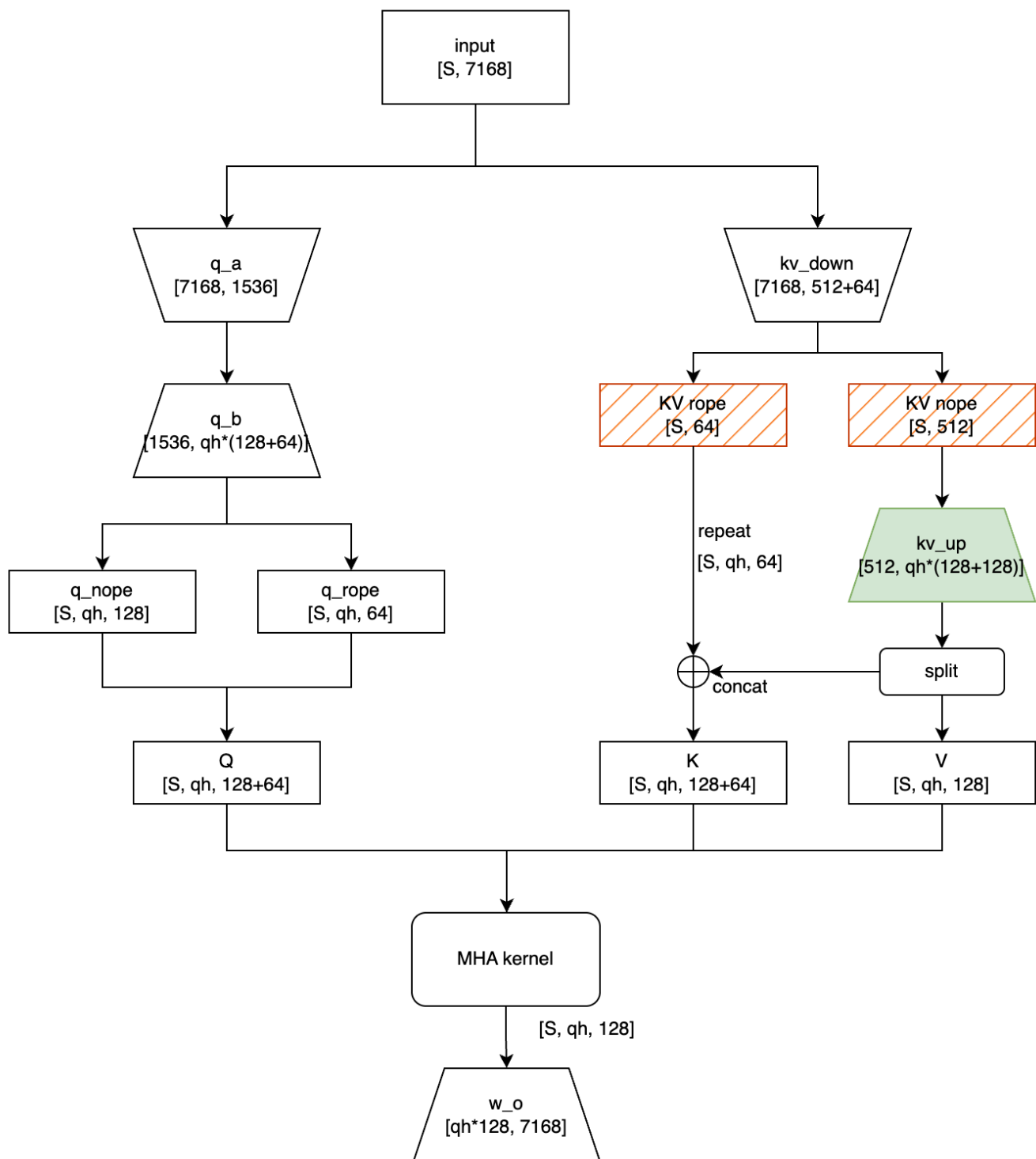
可是即使 MLA 使用得很频繁，感觉大家对 MLA 的计算过程的理解却有些参差。一说 MLA，都能说出矩阵吸收，什么 [prefill 模式⁺](#) 和 [decode 模式⁺](#)，但是具体的 prefill 和 decode 有什么不同，参数矩阵怎么设置，就有点云里雾里，甚至得出空间换时间这种有点似是而非的结论。

我认为这和现有的理解方式都习惯从矩阵视角阐述有一定关系，但是深度学习的负载通常是多维的，MLA 还要分 head，下标一多就容易乱。但我们都写过 torch 的嘛，tensor shape 就很清晰明了，再用图把它们串起来，建立一个算法到 torch 代码的对应关系，理解起来就容易多了。

本文就意图达到，看一眼流程图就了解 MLA 的计算过程，万一有一天忘了细节，再看一眼就能想起来。

Prefill 阶段





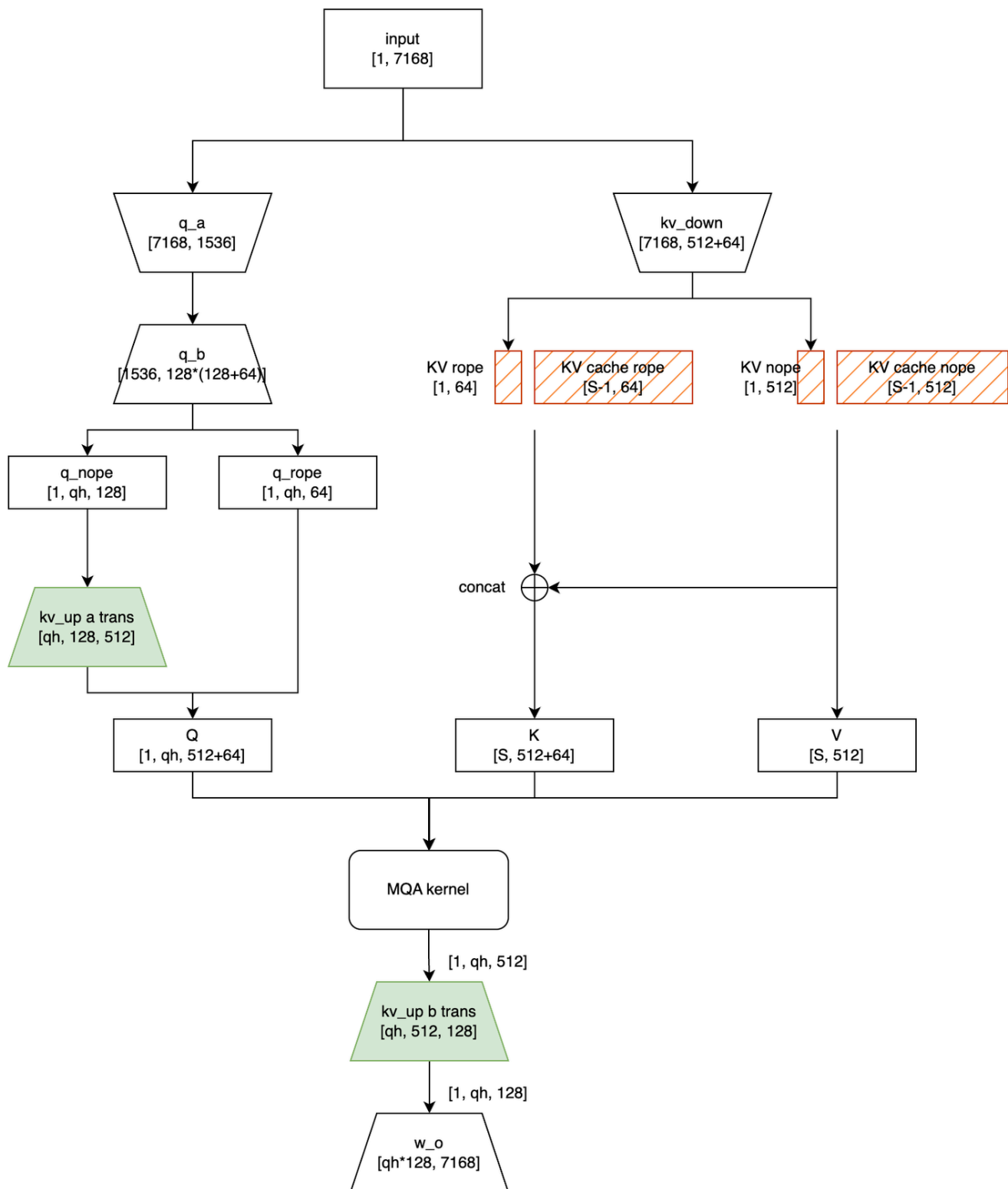
图中方框表示中间激活值，梯形表示模型参数，数字就是各张量的形状。设序列长度为 S ，头的数量为 qh 。dimension 维度就不用记号了，以免混乱，直接按照原论文的设置来。省略了 layerNorm 操作。

先降维再升维的操作在图中很明显，input 先后和 q_a , q_b 矩阵相乘，或是 input 先后和 kv_down , kv_up 矩阵相乘。红色斜线表示的是降维后的 KV rope, KV nope，就作为 KV cache 保存起来，以待后续的 Decode 阶段使用。

这里能看出一个“trick”，Q 经过位置编码的张量是多头的，但是 K 的是单头的，通过复制成多个头然后和升维后的张量 concat，构成 K 张量。这个 trick 在 Tri Dao 的 [GTA](#), [GLA](#) 中有专门讨论。

注意这里的 K 和 V 是不同的。由于升维后的 Q、K、V 都是多头的，接下来可以传入 MHA 的算子中进行运算，比如 [FlashAttention⁺](#)。

Decode 阶段



在 decode 阶段 1 次传入 1 个 token，设这个序列先前的 KV cache 长度是 S-1，当前 token 的 key 张量和 KV cache “拼”起来，构成形状为 [S, 512] 的张量。

这里多出了两个参数矩阵，kv_up a trans 和 kv_up b trans，是由原来的 kv_up 矩阵变化而来。这就是“矩阵吸收”的过程了。我们可以分 head 来看这个过程。

在 Prefill 阶段 KV nope 经过 kv_up，形状变化如下

```
[S, 512] @ [512, qh*(128+128)] -> [S, qh*(128+128)]
-> Reshape -> [S, qh, (128+128)] -> Split -> [S, qh, 128], [S, qh, 128]
```

@ 符号表示矩阵乘法，再经过 reshape 和 split 变成 K 和 V 张量。

从单个 head 视角来看 K 张量的形成，就是 $[S, 512] @ [512, 128] \rightarrow [S, 128]$ 。共有 qh 个 head，那么最后 K 的形状是 [qh, S, 128]，再 reshape 一下就是 [S, qh, 128] 和图中一致。

现在再从单个 head 视角看论文中的 attn 公式

$$\text{Softmax}((c_t^Q W_q)(W_k^T (c_t^{KV})^T)) c_t^{KV} W_v$$

$c_t^Q W_q$ 得到图中的 q_nope，单 token 单个 head 的形状是 [1, 128]， W_k 单个 head 的形状是 [512, 128]，那么转置后就是 [128, 512]。矩阵乘法符合结合律，可以去掉括号，先做 $c_t^Q W_q W_k^T$ 的计算，得到维度是 512。

即图中的 q_nope 经过 kv_up a trans 之后，得到 [1, qh, 512] 的形状。是每个 head 做了一个 $[1, 128] @ [128, 512] \rightarrow [1, 512]$ 的矩阵乘法。

这时可以发现，concat 上位置编码张量后，Q 和 K 的 head dim 维度相同，不过 Q 是多头的，K 是单头，那就可以采用 MQA+ 算子来做，只不过这是一个较为特殊的 MQA：

- K 的前 512 维度和 V 是同一个东西；
- 512 维度较大，其他的 attn 一般不会这么大

这应该也是 FlashMLA 算子所考虑的背景。

kv_up b trans 的变换同理上面的分析，MQA 出来的结果 per head 与其做矩阵乘法。

再回头看 Prefill 和 Decode 流程图，在 prefill 阶段做的是 MHA，而 decode 阶段居然能转换成 MQA 来做，这不仅是矩阵结合律（“吸收”）可以省计算量，算子换了更能达到提升计算密度的目的。这将在后文阐述。

计算量之谜

设传入当前阶段的 token 长为 x，先前的 KV cache 长度为 y，那么当前总序列长为 x+y。仅考虑矩阵乘法的计算量。如 A 矩阵形状 [M,K]，B 矩阵形状 [K, N]，那么 A 乘 B 的计算量为 $2*M*N*K$ ，忽略这个 2，因为后续比较也会约去。

不吸收的版本：(prefill 阶段认为 y=0)

$$\begin{aligned} \text{kv_up} &= (x+y)*512*qh(128+128) = qh * (x+y)*512*2*128 \\ \text{MHA} &= qh * (x(x+y) * 192 + x(x+y) * 128) \\ \text{all} &= qh * ((x+y)*512*2 + x(x+y) * 196 + x(x+y) * 128) \\ &= qh * ((x+y) * (512*2*128 + 196x + 128x)) \end{aligned}$$

吸收的版本：

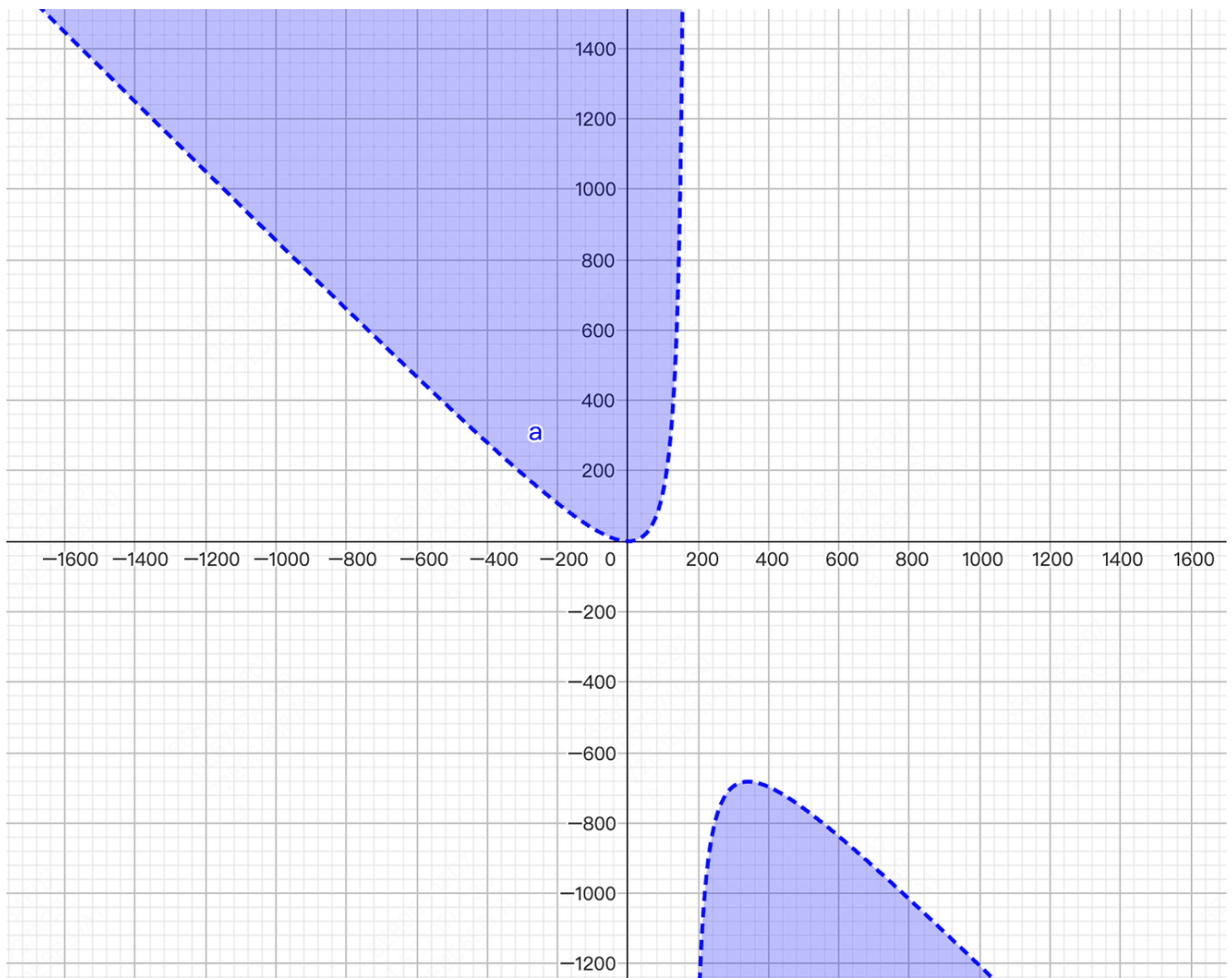
$$\begin{aligned} \text{kv_up_a} &= qh * x * 128 * 512 \\ \text{MQA} &= qh * (x(x+y) * 576 + x(x+y) * 512) \\ \text{kv_up_b} &= qh * x * 512 * 128 \\ \text{all} &= qh * (x * 128 * 512 + x * 512 * 128 + x(x+y) * 576 + x(x+y) * 512) \end{aligned}$$

什么时候不吸收的计算量会大于吸收的计算量，可以列出不等式

$$\begin{aligned} ((x+y) * (512*2*128 + 196x + 128x)) &> (x * 128 * 512 + x * 512 * 128 + x(x+y) * 576 + x(x+y) * 512) \\ \Rightarrow 512y &> 3x^2 + 3xy \end{aligned}$$

算这个计算量有啥用？因为现在的推理框架中并非只有 prefill 和 decode 两个阶段，还有传入的 token 数 > 1，同时 KV cache 也非空的中间态，比如 prefix cache、chunked-prefill 和投机解码。知道两者的计算量比较，就可以在知道 x 和 y 的取值后选择进行吸收或是不吸收。

这个不等式中 x 有二次项，但 y 只有一次项。直觉上，x 较小的时候不吸收的计算量大，而 x 较大的时候吸收的计算量大。实际也如此，不等式是一个如下图所示的双曲线区域，实际中 x 和 y 都 >= 0，因此只考虑双曲线上面的右边部分。



当 $x > 512/3 \approx 170$ 的时候，无论 y 的取值是多少，不吸收的计算量一定小于吸收的计算量，即此时应该做先升维再用 MHA 的版本。

对应到推理框架中的 chunked-prefill，常有 $x=2048$ ，而 $y > 2048$ 的情况，就应该选用不吸收的版本。而投机解码的 x 较小，就可以选用吸收的版本。

不过，序列长度 $(x+y)$ 可能很大，不吸收导致升维后的 K 、 V 激活值占用的显存也很大，这就是另一个故事了。

真正让计算更快

MQA 的计算密度更大，体现在理论计算量与访存量的比例更大，在 GPU 算子的实现中是怎么体现的呢？

Decode 阶段 Q 的形状是 $[1, qh, 576]$ 而 K 的形状是 $[S, 576]$ 。按 attn 公式的做法是 per head 计算 $Q \cdot K^T$ ，即

$[1, qh, 576] \rightarrow \text{transpose} \rightarrow [qh, 1, 576] @ [576, S] \rightarrow [qh, 1, S]$

每个 head 得到结果 $[1, S]$ 。但这里每个 head 的 K 是相同的，就可以做 $[1, qh, 576] @ [576, S] \rightarrow [1, qh, S]$ 的矩阵乘法。MNK 分别为 $qh, S, 576$ 。

Hopper 架构 GPU⁺ 中，wgmma 指令计算矩阵乘， M 最小为 64，而 Deepseek V3 模型中的 $qh=128 > 64$ ，就可以满满用上 wgmma 的算力。这也是 MQA 能让 decode 过程从访存密集型变成计算密集型的原因之一。

如果用了 TP，比如 $TP=8$ ，这时单张 GPU 上的 $qh=16 < 64$ 怎么办呢？一般就需在 M 维度 padding 上 0，依然运用 wgmma 指令，但这样存在无效运算，算力就没有打满了。

不过 Q 的长度可以 > 1 ，比如投机解码情况， Q 的形状是 $[x, qh, 576]$ ，那这时可以做 $[x * qh, 576] @ [576, S] \rightarrow [x * qh, S]$ 的矩阵乘法，结果加 mask，只要 $x * qh \geq 64$ 且 $x * qh \% 64 == 0$ 也可以用满算力。

而如果在 decode 阶段用 MHA，就没这么“幸运”了。[qh, 1, 192] @ [qh, 192, S] -> [qh, 1, S]，只能 per head 做 MNK 是 1, S, 192 的矩阵乘，算力被浪费了。

总结

本文用流程图展示了 MLA 的 prefill 和 decode 计算的过程，分析了吸收和不吸收的计算量差异，最后分析 MQA 算子怎么体现计算密度变大。期望让大家能够看一眼就理解 MLA 的计算过程。

本文是工作之外的业余时间所写，文中观点仅代表本人，不代表作者任职的机构。

编辑于 2025-10-14 09:08 · 北京

 赞同 340   14 条评论  分享  喜欢  收藏  申请转载 ...



未登录用户